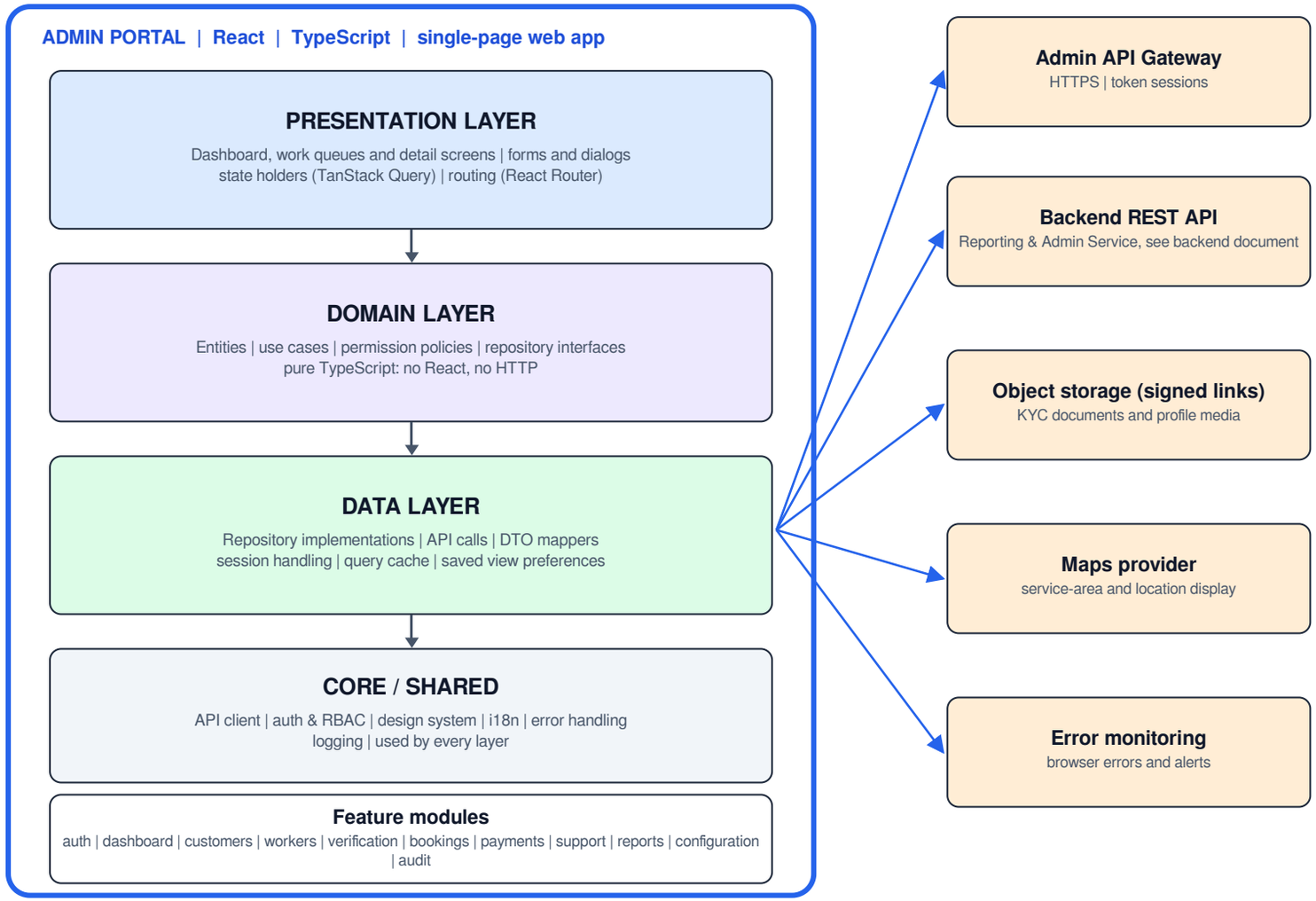
SEVANEST — ADMIN PORTAL FRONTEND ARCHITECTURE

React Architecture for the Web Admin & Operations Portal (desktop-first, responsive)



Calls flow downward. The Data layer implements Domain interfaces; the Domain layer never depends on React or HTTP.

One React codebase serves every admin role. Every backend call goes through the Admin API Gateway over HTTPS with token-based sessions. What a user can see is decided by permissions issued by the server, not by role names hard-coded in the UI. Only signed document links, map tiles and error reports are reached directly from the browser.

Presentation / UI · Domain / app logic · Data / cache / storage · Browser & external services · Security-sensitive · Core / shared / infrastructure

THE ARCHITECTURE IN SIMPLE TERMS

- **Presentation.** Queues, detail screens and forms with plain status labels. No API calls and no business rules here.
- **Domain.** The rules: entities and use cases such as ReviewVerification, ChangeBookingStatus and IssueRefund, plus permission policies.
- **Data.** Talks to the backend and to signed document links; turns API errors into domain failures.
- **Core / Shared.** API client, session, RBAC helpers, design system and logging used by every feature.
- **Backend.** The server is the source of truth for status, permissions, payments and the audit trail.

DECISION STATUS

Decision	Status
React web admin	Specified by the PRD
Role-based access control	Specified by the PRD
Audit logs for critical actions	Specified by the PRD
TypeScript, Vite, TanStack Query	Proposed
UI component library	Open item
Admin MFA	Open item
Supported browser matrix	Open item
Numeric performance budgets	Open item

ARCHITECTURAL DRIVERS

Goal	How the architecture addresses it
Run daily operations fast	Work queues (pending KYC, new requests, open tickets) with filters and one primary action per row
Trust before "verified"	Badge shown only when the server reports every configured check approved; per-check status stays visible
Role-based control	Menus, routes and buttons follow server-issued permissions; the server enforces every call
Protect sensitive data	Masked values in routine screens; documents through short-lived signed links; reveals are logged
Safe state changes	Admins see only the booking transitions the server permits; remarks and confirmation before irreversible actions
Concurrent operations	Version check on every edit; a clear conflict message instead of a silent overwrite
Auditability	Critical actions carry a reason and appear in the audit trail
Configurable business rules	Categories, areas, fees, plans, templates and roles are data edited in Configuration, not code
Data-heavy screens stay fast	Server-side paging, sorting and filtering; lazy-loaded charts and viewers
Maintainable by a small team	Layered, feature-first modules and typed API contracts (PRD team has one React developer)

PROPOSED

Items marked Proposed are architectural choices or assumptions not fixed by the PRD. They are confirmed during the architecture phase.

TECHNOLOGY STACK

Area	Choice	Notes
Framework / language	React, TypeScript	React per PRD; TypeScript proposed
Build tool (Proposed)	Vite	Fast builds, code splitting
Routing (Proposed)	React Router	Nested routes, guards, deep links
Server state (Proposed)	TanStack Query	Caching, paging, refetch, retry
UI state (Proposed)	Zustand (small stores)	Session, drawers, preferences
Forms (Proposed)	React Hook Form + Zod	Schema validation, field errors
HTTP client (Proposed)	axios with interceptors	Token, retry, error mapping
Data tables (Proposed)	TanStack Table	Server-side sort and filter
Charts (Proposed)	Recharts or ECharts	Dashboard and reports
UI components (Proposed)	Accessible library + tokens	Library to be confirmed
Session (Proposed)	In-memory access token + httpOnly cookie	No tokens in localStorage
Documents	Signed URLs, object storage	PRD: object storage for KYC
Maps / location	Google Maps or equivalent	PRD; service-area display
Localization (Proposed)	react-i18next	English in V1; ready for more
Monitoring	Cloud logs + alerts; browser error tracking	PRD; tool to be confirmed
Testing (Proposed)	Vitest, Testing Library, MSW, Playwright	Unit to end-to-end
CI/CD and hosting	Pipeline; static hosting + CDN on AWS or Azure	Environments by DevOps

Why this stack. React is specified by the PRD for the admin web. The other entries are proposed. API contract types and design tokens are shared with the Customer and Worker Apps so status labels and colours read the same in all three products.

Presentation / UI | Domain / app logic | Data / cache / storage | Browser & external services | Security-sensitive | Core / shared / infrastructure

LAYER RESPONSIBILITIES

Layer	Responsibility	Rules
Presentation	Pages, queues, detail screens, forms, route guards and state holders (hooks)	No API calls or business rules
Domain	Entities, use cases (ReviewVerification, ChangeBookingStatus, IssueRefund, AssignTicket), permission policies, repository interfaces	Pure TypeScript
Data	Repository implementations, API calls, DTOs, mappers, query keys	Implements domain interfaces; maps API errors to domain failures
Core / Shared	API client, session, RBAC helpers, design system, i18n, error and logging utilities	No feature-specific logic

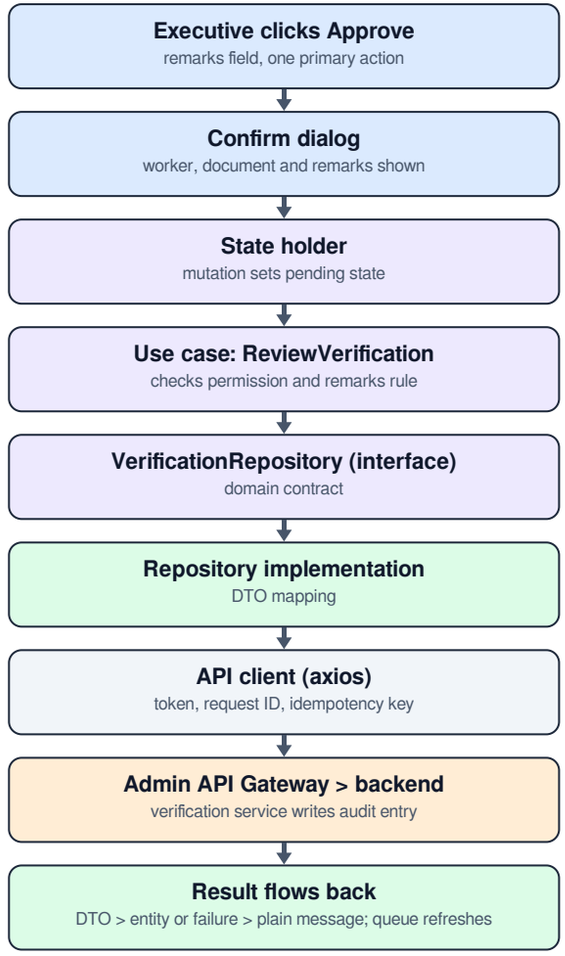
DEPENDENCY RULE



COMMUNICATION WITH THE BACKEND

- All calls go through the Admin API Gateway over HTTPS with token-based sessions.
- Each feature talks to its matching backend service area: Authentication, User/Profile, Worker Verification, Service/Category & Availability, Matching/Search, Booking, Payment, Notification, Support/Complaint, Reporting & Admin, and Audit & Security.
- Only signed document links, map tiles and browser error reports are reached directly from the browser.

EXAMPLE: APPROVE A VERIFICATION ITEM



Error handling. Failures return as domain errors (network, unauthorized, forbidden, validation, not found, conflict, server, unknown) and are shown as one short sentence with one next step. Technical detail goes to logs only.

Presentation / UI | Domain / app logic | Data / cache / storage | Browser & external services | Security-sensitive | Core / shared / infrastructure

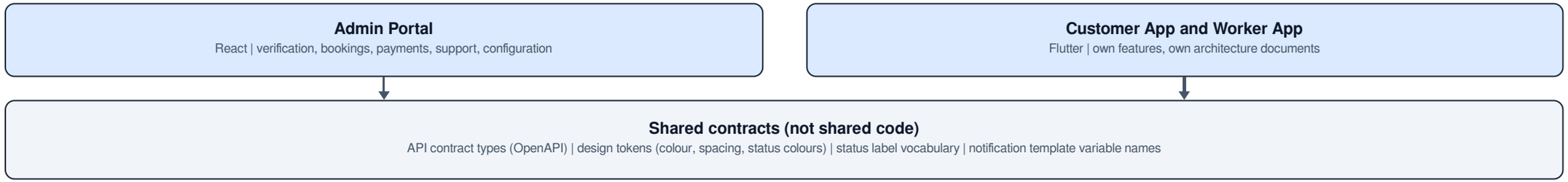
FEATURE-FIRST FOLDER LAYOUT PROPOSED

```
admin_portal/
  src/
    main.tsx            entry; environment config loaded first
    app/                providers, router, layout shell, theme
    core/               api, auth, rbac, storage, errors, logging
    shared/             design system, status labels, i18n
    features/
      auth/             login, session, password reset
      dashboard/        KPI cards, queue summaries
      customers/        list, profile, history, notes
      workers/          list, profile, documents, status
      verification/     queue, review, re-check
      bookings/         list, detail, match, transitions
      payments/         transactions, refunds, exports
      support/          tickets, assignment, SLA
      reports/          report views, exports
      configuration/    masters, fees, templates, roles
      audit/            audit trail viewer
      <feature>/{presentation, domain, data}
  tests/   e2e/
```

FEATURE MODULES AND MAIN SCREENS

Feature module	Main screens
auth	Login, Password reset, Session expired
dashboard	KPI cards (bookings, active workers, pending KYC, complaints, replacements, collections), queue summaries
customers	List, Profile and status, Saved addresses, Booking history, Complaints, Internal notes
workers	List, Profile (skills, availability, rating), Documents, Status: active / suspended / blacklisted
verification	Queue, Document review with remarks, Check status per type, Expiry and re-check list
bookings	New requests, Match workers, Interview / trial, Confirm / cancel, Replacement review, Timeline
payments	Transactions, Fee records, Refund requests and status, Invoice / reference export
support	Ticket list, Create ticket, Assign / escalate, Resolution SLA, Ticket timeline
reports	Demand, conversion, utilization and collection reports with export
configuration	Categories, Areas, Fees and plans, Notification templates, Roles and permissions
audit	Audit trail search by actor, action, record and date

SHARING WITH THE CUSTOMER AND WORKER APPS PROPOSED



Rule. Flutter and React cannot share code, so they share contracts. Admin-specific code (queues, review, refunds, configuration) stays inside the Admin Portal; the apps and the portal agree on the same API types, status names and design tokens.

Presentation / UI | Domain / app logic | Data / cache / storage | Browser & external services | Security-sensitive | Core / shared / infrastructure

APP FLOW

Login
email + password

→

Session check
token; MFA step is an open item

→

Landing page
first module the role may use

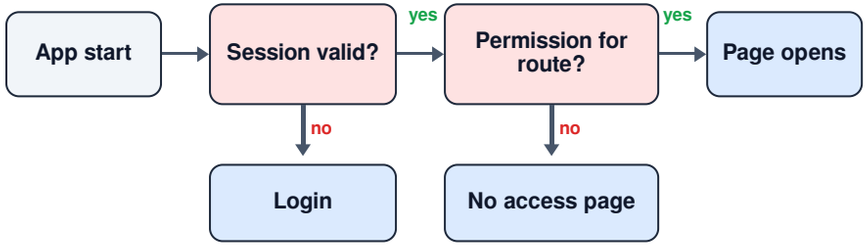
→

Main shell
sidebar navigation

MAIN SHELL: SIDEBAR MODULES (VISIBILITY DEPENDS ON ROLE)

Operations	People	Money & insight	Control
Dashboard and queues	Customer list and profile	Transactions	Categories and areas
Verification review	Worker list and profile	Refund requests	Fees, plans, templates
Booking pipeline	Documents (role-limited)	Reports	Roles and permissions
Support tickets	Suspend / blacklist	Exports	Audit trail

REDIRECT RULES



After sign-in the user returns to the page they asked for. A user without permission sees a plain No access page with a way back.

NAVIGATION RULES

- Router is declarative; routes are named and typed.
- Not logged in goes to Login; after sign-in the user returns to the page they asked for.
- Sidebar shows only the modules the user has permission for; a direct link to a hidden route shows a plain No access page.
- Records (booking, worker, ticket) have stable links so staff can share them.
- Filters, sort and page are kept in the URL, so a view survives refresh and can be bookmarked.
- Breadcrumbs and a visible back option on every detail screen.
- Global search finds customers, workers, bookings and tickets by ID, name or phone (Proposed).

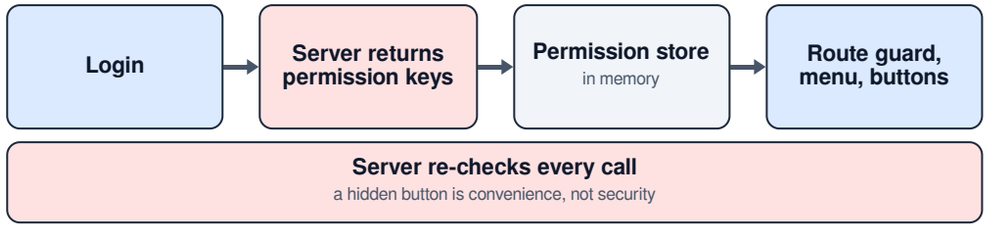
Presentation / UI | Domain / app logic | Data / cache / storage | Browser & external services | Security-sensitive | Core / shared / infrastructure

ROLE AND MODULE ACCESS (STARTING POINT) PROPOSED

Module	Super Admin	Operations Admin	Verification Exec.	Support Exec.
Dashboard	Manage	Manage	View	View
Customers	Manage	Manage	No access	View
Workers	Manage	Manage	View	View
Verification	Manage	Manage	Manage	No access
Sensitive documents	View	View	Manage	No access
Bookings	Manage	Manage	No access	View
Payments and refunds	Manage	Manage	No access	View
Support tickets	Manage	Manage	No access	Manage
Reports	Manage	View	No access	View
Configuration	Manage	No access	No access	No access
Audit trail	View	No access	No access	No access

Roles come from the PRD: Super Admin, Operations Admin, Verification Executive, Support Executive. The final matrix is set by the Super Admin in Configuration > Roles. The UI checks permission keys (for example **verification.review**), never role names.

HOW PERMISSIONS REACH THE SCREEN



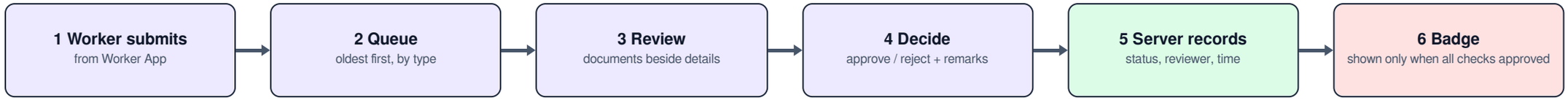
SENSITIVE DATA HANDLING

Data	Routine screens	When needed
Government ID / Aadhaar	Masked, last 4 only	Document viewer; role-limited; each view audited
KYC documents	Never in lists	Short-lived signed link; no public URL
Phone / address	Masked in lists	Reveal action with permission; logged (Proposed)
Emergency contact	Hidden	Visible based on policy and role
Payment reference	Masked reference	Full reference on payment detail for permitted roles
Internal notes	Latest note	Author and time on every note

Two checks, always. The UI hides what a user may not use, and the server refuses what the user may not do. The PRD requires role restrictions on sensitive screens, restricted KYC access, masked values in routine screens and no public exposure of full identity documents.

Presentation / UI | Domain / app logic | Data / cache / storage | Browser & external services | Security-sensitive | Core / shared / infrastructure

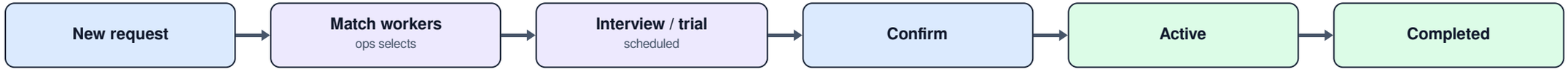
VERIFICATION JOURNEY: A FEW STEPS, ONE PURPOSE EACH



Checks follow the PRD: mobile, identity, address, emergency contact, previous employment, police verification status. Customers see a badge only after an authorised user approves the configured checks.

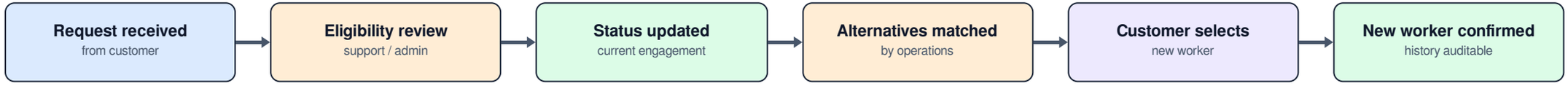


BOOKING PIPELINE (ADMIN SIDE)



Cancel and Replace branch from the statuses the server allows. Each booking shows only the actions the server permits for its status and the user's role; the portal never decides transitions itself.

REPLACEMENT JOURNEY



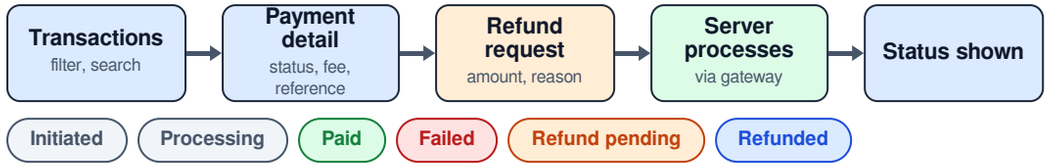
RULES IN THE PORTAL

- A worker shows as Verified only when the server reports all configured checks approved; per-check status stays visible.
- Reject, cancel, suspend and blacklist need remarks; remarks are saved with actor and time.
- Booking changes are limited to permitted states; invalid actions are not offered.
- Previous engagements stay visible in booking history after a replacement.

Why this design. The PRD says a worker is never presented as fully verified until checks are approved by an authorised user, and that every change stays auditable. Server-driven actions and mandatory remarks keep both true.

Presentation / UI | Domain / app logic | Data / cache / storage | Browser & external services | Security-sensitive | Core / shared / infrastructure

PAYMENTS AND REFUNDS



- No gateway SDK or secret in the portal; payment state comes from the server.
- Refunds are server actions with an idempotency key, remarks and confirmation.
- Invoice and reference exports are generated by the server, masked, and audited.

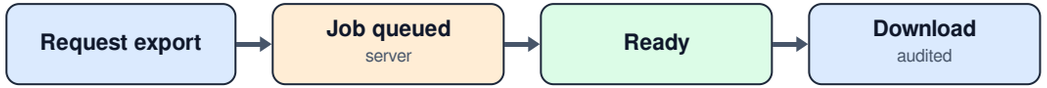
SUPPORT TICKETS



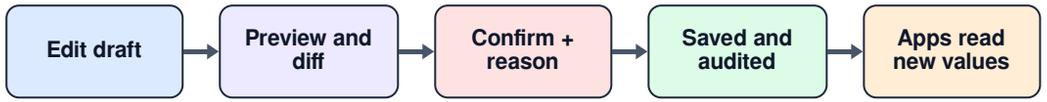
- Resolution SLA timer and breach flag come from the server.
- Complaint history shown on the customer and worker profile.

REPORTS

New customers | Worker verification conversion | Requests vs confirmed | Category demand | Area demand | Worker utilization | Interview-to-confirm | Cancellations | Replacements | Collections | Ticket SLA | Top-rated / repeat



CONFIGURATION MASTERS



Categories | Areas | Fees | Plans | Notification templates | Roles

- Fee values are configurable here, as the PRD requires; new categories need no app redesign.
- Templates cover push, SMS and optional WhatsApp, with variable checks.
- Role edits take effect at the next permission refresh.

Same rule on both sides. The server owns payment, refund, ticket and configuration state, verifies results and keeps the audit record. The portal collects the decision, shows the status the server reports, and never calculates it.

Presentation / UI | Domain / app logic | Data / cache / storage | Browser & external services | Security-sensitive | Core / shared / infrastructure

DESIGN RULES (VALUES SUBJECT TO DESIGN REVIEW)

Rule	Standard (Proposed)
Queue first	Each module opens on a filtered list with counts; one dominant action per row or detail
Dense but readable	Compact tables, clear row actions; usable at 100% zoom on a 1366 x 768 laptop
Targets	At least 32 px on desktop, 44 px on touch tablets
Icons with labels	Every icon has text or a tooltip and an accessible name
Contrast and colour	Status never relies on colour alone: text and icon accompany colour
Keyboard	Full keyboard use, visible focus, dialogs trap focus
Forgiving input	Confirm irreversible actions; warn on unsaved changes; remarks on reject, cancel and suspend
Tables	Server-side sort, filter and paging; filters kept in the URL
Trust cues	Verification badge and rating summary on every worker row and profile
Responsive	Desktop-first; tablet supported; phone shows dashboard and tickets read-only
Shared look	Same design tokens as the Customer and Worker Apps

STATUS LABEL MAPPING PROPOSED

Area	Backend status (examples)	Admin label
Verification	Pending	Pending review
Verification	In review	In review
Verification	Approved / Rejected	Approved / Rejected
Verification	Expiry due	Re-check due
Worker	Suspended / Blacklisted	Suspended / Blacklisted
Booking	Requested	New request
Booking	Interview / trial scheduled	Interview set
Booking	Confirmed	Worker confirmed
Booking	Active	Service active
Booking	Replacement requested	Replacement review
Payment	Success / Failed	Paid / Payment failed
Payment	Refund pending	Refund pending
Ticket	Escalated / SLA breached	Escalated / SLA breached

One mapping class converts a backend status into a short label, an icon, a colour and the next permitted action. Unknown statuses fall back to a generic label instead of failing.

REUSABLE COMPONENTS (STATELESS, DRIVEN BY DATA)

Data table | Filter bar | Status badge | Verification badge | Detail drawer

Confirm dialog with remarks | Document viewer | Status timeline | KPI card

Permission gate | Date range picker | Audit entry | Empty and error views

HOW LABELS LOOK ON SCREEN

Pending review | Approved | Rejected | Interview set | Paid | Payment failed

Escalated | SLA at risk

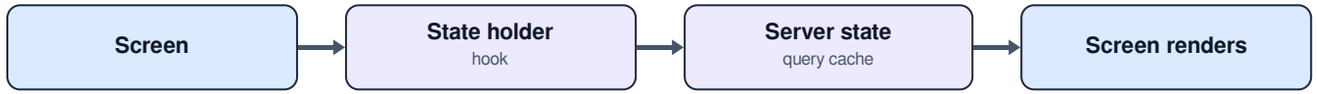
Why stateless. Stateless components keep the UI testable and consistent with the apps. They support the PRD aims of simple, structured operations and plain status labels, and they let each module reuse the same table, filter and dialog behaviour.

Presentation / UI | Domain / app logic | Data / cache / storage | Browser & external services | Security-sensitive | Core / shared / infrastructure

STATE BY FEATURE (REACT QUERY + ZUSTAND, PROPOSED)

Feature	State held	Lifetime
Session (global)	User, permission keys, idle timer, language	App lifetime
Dashboard	KPI summaries, queue counts	Cached; periodic refetch
Customers / Workers	List filters, selected record, notes	URL + cache
Verification	Queue filters, open review, draft remarks	Feature
Bookings	Filters, selected booking, allowed actions	Feature
Payments	Transaction filters, refund draft	Feature
Support	Ticket filters, assignment, SLA view	Feature
Reports	Date range, filters, export job	Feature
Configuration	Draft changes before save	Until saved or discarded

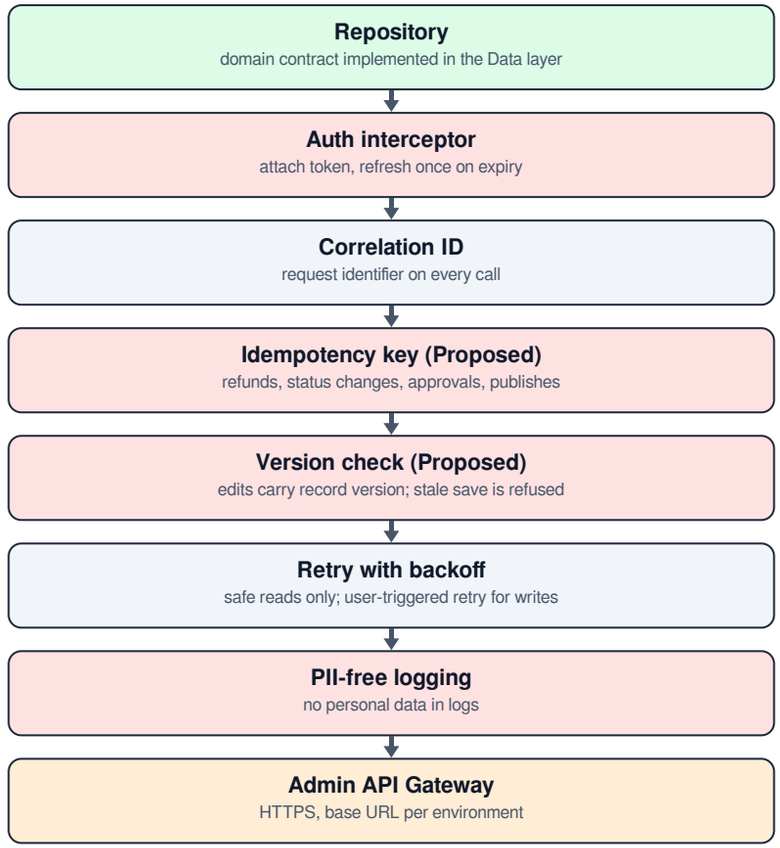
HOW A SCREEN RENDERS



Every async screen has: loading · data · empty · error · no access

- Server data lives in the query cache; local UI state stays small.
- Filters, sort and page live in the URL.
- After a change, related queries are refetched so lists never show stale status.
- The server is the source of truth for status, permissions and payments.

API CLIENT: EVERY CALL PASSES THE SAME PIPELINE



FAILURES MAP TO DOMAIN ERRORS

Network · Unauthorized · Forbidden · Validation · Not found · Conflict · Server · Unknown

Repository interfaces live in the Domain layer; implementations live in the Data layer. API responses are mapped from DTOs to domain entities, so screens never see raw API shapes.

Why version checks. Several admins work at once (the PRD calls for role-based concurrent operations). A version check on every edit means a stale save is refused with a clear message, instead of silently overwriting a colleague's change.

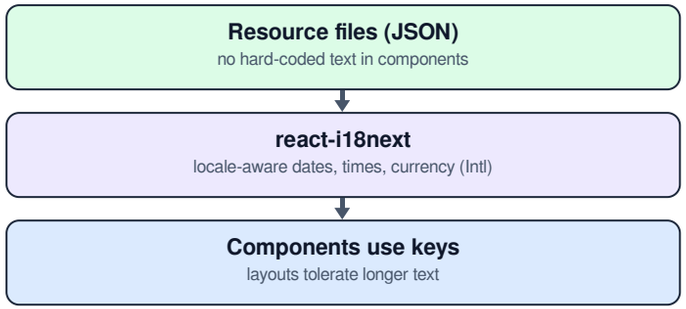
Presentation / UI · Domain / app logic · Data / cache / storage · Browser & external services · Security-sensitive · Core / shared / infrastructure

WEAK NETWORK BEHAVIOUR PROPOSED

Data / action	Behaviour
Lists and queues	Show last loaded data with a "last updated" time; refresh on focus and on demand
Approvals, status changes, refunds	Require network; clear message and retry; never silently dropped or repeated
Reference data (categories, areas)	Cached in memory and refreshed on load
Forms in progress	Draft kept if the session expires; restored after sign-in
Session token	Short-lived; refreshed quietly; idle timeout signs out

Scope. V1 requires graceful failure on slow or unstable office networks. A full offline mode is not required.

LOCALIZATION READINESS



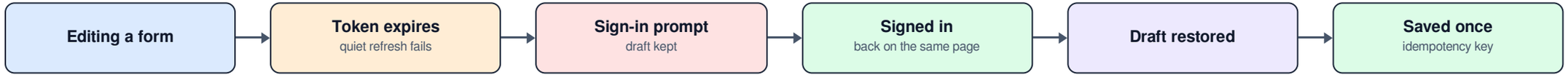
- Admin UI ships in English in V1 (assumption: the PRD Phase 2 multi-language plan targets the apps).
- Dates in IST; money in INR with standard formatting.
- Notification templates are data and can hold per-language text.
- Worker names, areas and skills appear as provided by the server.

PERFORMANCE APPROACH

Area	Approach
First load	Route-level code splitting; charts and document viewer loaded on demand
Tables	Server-side paging, sort and filter; virtual rows for long lists
Search	Debounced input; stale requests cancelled
Documents and images	Thumbnails in lists; full file only when opened
Caching	Query cache with refresh in the background
Bundle size	Tree shaking; size check in the pipeline
Devices	Standard office laptops and tablets; agreed browser matrix
Memory	Release viewers and listeners when screens close

Open item. The PRD sets qualitative targets only. Numeric performance budgets are to be agreed before performance testing.

WHAT AN ADMIN SEES WHEN THE SESSION EXPIRES MID-FORM



Principle. Writes are never dropped silently: the admin always sees success or a plain message with a next step. Reads fall back to the last loaded data with a "last updated" indicator.

Presentation / UI | Domain / app logic | Data / cache / storage | Browser & external services | Security-sensitive | Core / shared / infrastructure

SECURITY ARCHITECTURE

Concern	Approach
Transport	HTTPS for all traffic; certificate validation; HSTS (Proposed)
Authentication	Password sign-in with token-based session; MFA is an open item
Authorization	Server-issued permissions; route guards and gates in the UI; server re-checks
Sensitive data	Masked in routine screens; documents by short-lived signed links; reveals logged
Session	Tokens in memory plus httpOnly cookie, none in localStorage; idle timeout (Proposed)
Browser hardening	Content Security Policy, no inline scripts, frame protection, dependency audit in CI (Proposed)
Secrets	No API secrets in the bundle; only public settings injected per environment
Audit	Reason captured for critical actions; server audit log for verification, payment and user-status changes; Super Admin viewer
Input and files	Schema validation; output escaped; attachment type and size checked before upload
Exports	Respect masking and permissions; every export audited
Account state	Suspended admin accounts are signed out with a clear message
Privacy and consent	Terms, privacy and consent content follows legal review; data retention is an open item

WHERE DATA LIVES IN THE BROWSER

Memory
access token, permissions, drafts

httpOnly cookie
refresh credential only

Local storage
UI preferences only, no personal data

Logs / error reports
no personal data

INTEGRATIONS

Integration	Frontend approach
Object storage (documents)	Signed link requested per view; expires quickly; inline viewer with no public URL
Maps / location	Service-area display and area entry in Configuration; manual entry if the map is unavailable
Notifications	Edit push, SMS and optional WhatsApp templates; view delivery status from server records
Payment gateway	No SDK in the portal; status and refunds through the backend
Error monitoring	Browser errors captured with release version; no personal data

OBSERVABILITY AND ERROR HANDLING

- Browser error capture and non-fatal error reporting in production, with source maps kept private.
- Structured logging without personal data; correlation ID links a screen error to the backend call.
- Analytics events (Proposed): queue opened, verification decided, booking status changed, refund issued, ticket resolved, report exported.
- Error messages are short, plain and give one next step; technical detail goes to logs only.

Presentation / UI · Domain / app logic · Data / cache / storage · Browser & external services · Security-sensitive · Core / shared / infrastructure

TESTING STRATEGY

Level	Scope	Tooling
Unit	Use cases, permission policies, mappers, validators, status mapping	Vitest
Component	Design-system components and key screens in loading, empty, error and no-access states	Testing Library
Integration	Login, verification decision, booking transition, refund and configuration edit against a mocked API	MSW + Vitest
End-to-end	Role restrictions, verify, booking state change, payment status, refund, replacement, ticket	Playwright
Accessibility	Keyboard and screen-reader checks on key screens	axe + manual
Browser	Agreed supported browser matrix	Cross-browser runs
Non-functional	Performance, security headers, dependency checks	Profiling, audit tools
UAT	Critical workflows per acceptance criteria with the business team	Manual

ENVIRONMENTS

Environment	Backend	Purpose
dev	Development	Daily development
stage	Staging	QA and UAT
prod	Production	Live operations

Environment settings are injected at deploy time; no secrets live in the bundle.

CI/CD AND RELEASE



Versioned builds with a changelog; static files served from storage behind a CDN. Production monitoring and error alerts are active at release; a previous build can be restored quickly.

ARCHITECTURE SUMMARY

Item	Summary
Framework	React + TypeScript single-page app (PRD: React admin web)
Architecture	Layered, feature-first modules; contracts shared with the apps
State / navigation	TanStack Query + Zustand | React Router with permission guards (Proposed)
Access	Server-issued permissions; masked data; signed document links
Operations UI	Queues, remarks, server-permitted actions, plain status labels
Localization	i18n-ready; English in V1

OPEN OR PROPOSED ITEMS

- Numeric performance budgets
- Admin MFA
- Supported browser matrix
- UI component library
- Cookie vs token session design with the backend
- Data retention rules
- Reveal-sensitive-data policy
- Final role and permission matrix
- Export formats and size limits
- Error monitoring tool

The SevaNest Admin Portal is designed as a layered, feature-first React application: operations staff can verify workers, manage bookings, payments and support, and configure the platform with plain status labels, while the server remains the source of truth and every critical action stays auditable.

Presentation / UI | Domain / app logic | Data / cache / storage | Browser & external services | Security-sensitive | Core / shared / infrastructure